

1. processid

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("My Process ID (PID) = %d\n", getpid());

    return 0;
}
```

```
akhil@Akhil:~/OS$ gcc getpid.c -o gitpid
akhil@Akhil:~/OS$ ./gitpid
My Process ID (PID) = 556
akhil@Akhil:~/OS$ |
```

2.creating child using fork()

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
    }
    else
    {
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());
    }

    return 0;
}
```

```
akhil@Akhil:~/OS$ gcc fork.c -o fork
akhil@Akhil:~/OS$ ./fork
Parent Process
PID = 600
Child Process
PID = 601
akhil@Akhil:~/OS$ |
```

3. Parent Waits for Child (wait())

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process Executing...\n");
        sleep(3);
        printf("Child Finished\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Process Continues\n");
    }

    return 0;
}
```

```
akhil@Akhil:~/OS$ gcc childwait.c -o childwait
akhil@Akhil:~/OS$ ./childwait
Child Process Executing...
Child Finished
Parent Process Continues
akhil@Akhil:~/OS$ |
```

4. Multiple Child Processes

```

#include <stdio.h>
#include <unistd.h>

int main()
{
    for(int i=1;i<=3;i++)
    {
        if(fork()==0)
        {
            printf("Child %d PID = %d\n", i, getpid());
            return 0;
        }
    }

    return 0;
}

```

```

akhil@Akhil:~/OS$ gcc multiple.c -o multiple
akhil@Akhil:~/OS$ ./multiple
Child 1 PID = 726
Child 2 PID = 727
Child 3 PID = 728
akhil@Akhil:~/OS$ |

```

5. Execute linux command using execlp()

```

#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Before exec()\n");

    execlp("ls", "ls", "-l", NULL);

    printf("This will not execute.\n");

    return 0;
}

```

```

akhil@Akhil:~/OS$ gcc execlp.c -o execlp
akhil@Akhil:~/OS$ ./execlp
Before exec()
total 196
-rwxr-xr-x 1 akhil akhil 16080 Aug 12 06:37 childwait
-rw-r--r-- 1 akhil akhil 336 Aug 12 06:37 childwait.c
-rwxr-xr-x 1 akhil akhil 16000 Aug 12 06:39 execlp
-rw-r--r-- 1 akhil akhil 179 Aug 12 06:39 execlp.c
-rwxr-xr-x 1 akhil akhil 16080 Aug 12 06:36 fork
-rw-r--r-- 1 akhil akhil 297 Aug 12 06:36 fork.c
-rw-r--r-- 1 akhil akhil 122 Aug 12 06:35 getpid.c
-rwxr-xr-x 1 akhil akhil 16000 Aug 12 06:35 gitpid
-rwxr-xr-x 1 akhil akhil 15952 Aug 7 12:49 hello
-rw-r--r-- 1 akhil akhil 85 Aug 7 12:49 hello.c
-rw-r--r-- 1 akhil akhil 19 Jul 29 06:03 info.txt
-rw-r--r-- 1 akhil akhil 12 Aug 6 04:14 input.txt
-rw-r--r-- 1 akhil akhil 19 Jul 29 06:03 klu.txt
-rwxr-xr-x 1 akhil akhil 16040 Aug 12 06:39 multiple
-rw-r--r-- 1 akhil akhil 238 Aug 12 06:38 multiple.c
-rwxr-xr-x 1 akhil akhil 16056 Aug 7 12:52 name
-rw-r--r-- 1 akhil akhil 134 Aug 7 12:53 name.c
-rw-r--r-- 1 akhil akhil 12 Aug 6 04:14 output.txt
-rwxr-xr-x 1 akhil akhil 16000 Aug 7 12:54 process
-rw-r--r-- 1 akhil akhil 97 Jul 30 04:06 processId.c
-rwxr-xr-x 1 akhil akhil 16168 Aug 6 04:13 week1_2
-rw-r--r-- 1 akhil akhil 414 Aug 6 04:02 week1_2.c
akhil@Akhil:~/OS$ |

```

6. Parent and Child Executing Concurrently

```

#include <stdio.h>
#include <unistd.h>

int main()
{
    fork();

    for(int i=1;i<=5;i++)
    {
        printf("PID: %d Count: %d\n", getpid(), i);
        sleep(1);
    }

    return 0;
}

```

```

akhil@Akhil:~/OS$ gcc parentchild.c -o parentchild
akhil@Akhil:~/OS$ ./parentchild
PID: 804 Count: 1
PID: 805 Count: 1
PID: 804 Count: 2
PID: 805 Count: 2
PID: 805 Count: 3
PID: 804 Count: 3
PID: 804 Count: 4
PID: 805 Count: 4
PID: 804 Count: 5
PID: 805 Count: 5
akhil@Akhil:~/OS$ |

```

7. Zombie Process

```

#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Exiting\n");
        return 0;
    }
    else
    {
        sleep(15);
        printf("Parent Finished\n");
    }

    return 0;
}

```

```

akhil@Akhil:~/OS$ gcc zombie.c -o zombie
akhil@Akhil:~/OS$ ./zombie
Child Exiting
Parent Finished

```

8. OrphanProcess

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid > 0)
    {
        printf("Parent Exiting\n");
        return 0;
    }
    else
    {
        sleep(5);
        printf("Child PID : %d\n", getpid());
        printf("New Parent PID : %d\n", getppid());
    }

    return 0;
}
```

```
akhil@Akhil:~/OS$ gcc orphan.c -o orphan
```

```
akhil@Akhil:~/OS$ ./orphan
```

```
Parent Exiting
```